

Source code is wrong

Rust & Python Codes are uploaded in [Rust-projects/text_wizard](#)

Python & Rust Integration Guide

A complete step-by-step workflow using **PyO3** and **Maturin**

Step 1: Tools Installation (Rust & Maturin)

First, ensure Rust and Maturin are installed on your system. Rust provides the compiler, and Maturin acts as the build system bridging Python and Rust.

- **Install Rust:** Install via `rustup` from the official website.
- **Install Maturin:** Open your Python virtual environment and run:

```
pip install maturin
```

Step 2: Initialize the Project

Instead of manually configuring CMake or `setup.py`, Maturin sets up the entire project boilerplate in one command.

```
maturin new text_wizard
```

Select `pyo3` when prompted. This creates a directory with two main files:

- `Cargo.toml`: Rust package manager config (includes PyO3 dependency).
- `src/lib.rs`: The main Rust source file.

Step 3: Write Core Logic in Rust

Open `src/lib.rs`. We write a function to count unique words and expose it to Python using PyO3 macros. Notice the absence of manual memory management or pointer handling.

```
use pyo3::prelude::*;
use std::collections::HashSet;

// 1. Core Function (Exposed to Python via #[pyfunction])
#[pyfunction]
fn count_unique_words(text: String) -> usize {
    let words: HashSet<&str> = text.split_whitespace().collect();
    words.len()
}

// 2. Module Initialization (Python module name: 'text_wizard')
#[pymodule]
fn text_wizard(_py: Python, m: &PyModule) -> PyResult<()> {
    m.add_function(wrap_pyfunction!(count_unique_words, m)?);
    Ok(())
}
```

Step 4: Build the Machine Code (Wheel)

Use Maturin to compile the Rust code into an optimized machine binary and package it into a standard Python `.whl` file.

```
maturin build --release
```

Maturin automatically detects the OS architecture, compiles the code into a shared library, and wraps it. The output will be located in the `target/wheels/` directory.

Step 5: Install and Use in Python

Install the generated wheel file directly using `pip`.

```
pip install target/wheels/text_wizard-0.1.0-cp310-cp310-win_amd64.whl
```

Note: During development, you can simply run `maturin develop --release` to instantly compile and install the module into your active Python environment.

Usage in Python (main.py):

```
import text_wizard
import time

big_text = "rust is fast python is easy " * 100000

start_time = time.time()
unique_count = text_wizard.count_unique_words(big_text)

print(f"Unique words found: {unique_count}")
print(f"Time taken: {time.time() - start_time} seconds")
```

Why is this approach superior?

- **Zero Boilerplate:** Maturin handles the entire build process natively.
- **Safety & Speed:** Rust guarantees memory safety while matching C/C++ execution speeds.
- **Modern Ecosystem:** This architecture powers modern high-performance Python tools like Polars, Ruff, and Pydantic V2.